IP/Port/URL Allow-List & Egress Filtering Management for Microsoft 365 & Intune Endpoints

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary 4

The situation . . . 4

The solution . . . 4

What is included . . . 4

The outcome . . . 4

Important notes . . . 5

Solution Overview 5

The problem this solves . . . 5

The Microsoft required-endpoints model . . . 5

The allow-list lifecycle . . . 7

Deployment model . . . 7

Requirements Guide 8

R1. A proxy or firewall you administer, with a default-deny egress posture . . . 8

R2. The Microsoft required-endpoints source (endpoints web service reachable) . . . 8

R3. Change control and a documented rollback . . . 9

R4. A pilot scope (small, representative device set) . . . 9

R5. Entra / Intune reachability from the management workstation . . . 10

Implementation Guide 10

Phase 0 — Assess (prerequisites and baseline) . . . 11

Phase 1 — Derive the required set (Configure, part 1) . . . 11

Phase 2 — Build the allow-list / proxy exceptions (Configure, part 2) . . . 12

Phase 3 — Pilot (apply to a limited scope) . . . 13

Phase 4 — Validate reachability and real workflows . . . 13

Phase 5 — Expand to the fleet . . . 13

Phase 6 — Keep current (operate the loop) . . . 14

Testing Guide 14

Gate T1 — Required-set derived correctly . . . 14

Gate T2 — Per-endpoint reachability (the core reachability gate) . . . 15

Gate T3 — Coverage / gap report clean . . . 15

Gate T4 — Enrollment / Autopilot succeeds . . . 15

Gate T5 — Policy / compliance sync succeeds . . . 15

Gate T6 — Update / app delivery succeeds . . . 15

Gate T7 — Defender cloud protection reachable (if in scope) . . . 15

Gate T8 — Drift check operates . . . 15

Overall exit criteria (before Expand) . . . 16

Troubleshooting Guide 16
Diagnostic method (use for every symptom) . . . 16
Symptom: Enrollment or Autopilot stalls / errors . . . 16
Symptom: Device shows enrolled but policies never sync / stay "pending" . . . 17
Symptom: Windows Update or app install fails / hangs . . . 17
Symptom: Defender cloud protection disconnected . . . 17
Symptom: Everything intermittently breaks after "no change on our side" . . . 18
Symptom: Validation passes but a workflow still fails . . . 18
Script-level issues . . . 18
Best Practices 19
Change control . . . 19
Derive, never hardcode . . . 19
Drift review . . . 20
SSL inspection and least privilege . . . 20
Documentation and handover . . . 20
Deliverables Summary 20
Documents . . . 21
PowerShell scripts (`.\scripts\`) . . . 21
Named objects you will create (verbatim) . . . 22
What "done" looks like . . . 22
Important notes . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

The situation

Tight egress filtering — denying outbound internet traffic by default and allowing only explicit destinations — is a required control in regulated and security-conscious environments. But Microsoft 365, Intune, Windows Autopilot, Microsoft Defender, and Microsoft Entra ID are cloud services: a managed device must reach a **specific, Microsoft-published set of endpoints** to enroll, sync policy, receive updates, and stay protected. When the allow-list misses even one required endpoint, the result is not a clear error — it is **enrollment that stalls, policies that never sync, and updates that fail silently**, with no obvious network cause. And because Microsoft changes the endpoint list regularly, a list that was correct last quarter drifts out of date on its own.

The solution

A repeatable operating method — plus five PowerShell scripts — to **administer the IP/Port/URL allow-lists and proxy/firewall exceptions** for Microsoft's required endpoints, built around one principle: **derive the required set live from Microsoft's published source, never hardcode it.**

The method runs as a lifecycle: **derive** the current required IP/Port/URL set from the Microsoft 365 IP/URL web service (for your tenant's instance) plus the documented Intune, Autopilot, and Defender endpoints; **build** named allow-list entries and proxy exceptions; **validate** that every required destination is actually reachable and that real workflows (enrollment, sync, update) succeed on a **pilot**; **expand** to the fleet; and **keep current** with a scheduled drift check that flags added or retired endpoints before they cause failures. It follows a staged model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with a pass/fail gate at every step.

What is included

- **Solution, Requirements, and Implementation guides** — the why, the prerequisites (each with how to

verify and how to implement if missing), and the exact ordered steps.

- **Five PowerShell scripts** — required-endpoint + filter-state discovery (with drift detection),

per-endpoint reachability validation, an allow-list coverage/gap report, safe allow-list update generation (preview first), and a pilot target-group builder. The read-only scripts change nothing; the change-generating scripts preview by default.

- **Testing and Troubleshooting guides** — pass/fail gates (endpoint reachable; enrollment, sync, and

update succeed) and a symptom-to-missing-endpoint map.

- **Best Practices** — change control and drift review to keep the list healthy.

The outcome

Cloud management that works reliably through strict egress filtering: the correct IP/Port/URL allow-lists and proxy exceptions, derived from Microsoft's live published set, validated on a pilot, and kept current —

delivered as a repeatable method and scripts you keep and reuse, at a fraction of the cost of an outside engagement.

Important notes

- **Vendor-neutral on the proxy/firewall device** — the method specifies what to allow and how to name

it; the specific console steps are deferred to that vendor's documentation.

- **No hardcoded Microsoft IP addresses** — the required set is always derived live, because

Microsoft's endpoints change over time.

Solution Overview

The problem this solves

Regulated and security-conscious organizations run **tight egress filtering**: outbound internet traffic is denied by default and only explicitly allowed destinations (by IP address, port, or URL) are permitted through a proxy or firewall. This is a sound security posture — but it collides with a fact about modern device management: **Microsoft 365, Intune, Windows Autopilot, Microsoft Defender, and Microsoft Entra ID are cloud services.** A device that is being enrolled, managed, protected, and updated must reach a **specific, published set of Microsoft endpoints** to do any of it.

When the allow-list is incomplete, the failure is quiet and confusing:

- **Autopilot / enrollment stalls** — a new device sits at a provisioning screen, or enrollment

errors out, because it cannot reach the enrollment or device-registration endpoints.

- **Policies never sync** — Intune shows a device as enrolled, but configuration, compliance, and app

policies never land because the device management check-in endpoint is blocked.

- **Updates fail silently** — Windows Update / delivery of content fails or hangs because content and

update endpoints are blocked.

- **Defender / Entra features degrade** — Defender cloud protection, or Entra sign-in / device

registration, fail because their endpoints were never added.

None of these produce an obvious "the firewall blocked you" message on the device. They look like random breakage. The fix is not to loosen the firewall — it is to **administer the allow-list precisely**: derive the required destinations, allow exactly those, validate that they are reachable, and keep the list current as Microsoft changes it.

The Microsoft required-endpoints model

Microsoft publishes the network destinations its cloud services need, and — critically — it publishes them in a **machine-readable, versioned form** so you never have to guess or hardcode addresses.

The Microsoft 365 IP Address and URL web service

Microsoft operates a public **Microsoft 365 IP Address and URL web service** (commonly called the "endpoints web service"). It returns, as structured data, the current set of URLs, IP address ranges, and ports that Microsoft 365 uses, organized into **endpoint sets**. Each endpoint set carries attributes you use to build a correct, minimal allow-list:

- **id** — a stable numeric identifier for the endpoint set.
- **serviceArea** — which workload it belongs to (for example Exchange, SharePoint, Skype/Teams, or the common/identity area that Entra and other shared services use).
- **urls** — the URLs (often with wildcards) the service uses.
- **ips** — the IPv4/IPv6 CIDR ranges the service uses.
- **tcpPorts / udpPorts** — the ports required.
- **category** — **Optimize**, **Allow**, or **Default**. Optimize and Allow are the endpoints required for the service to work correctly and should always be permitted; Default endpoints are broadly used and can typically be sent to the general internet path.
- **required** — a true/false flag marking endpoints that are mandatory for connectivity.
- **expressRoute**, **notes** — routing hints and human-readable context.

The service also exposes a **version endpoint** so you can detect when the published set has changed without re-downloading everything. **Every endpoint set is keyed to an "instance"** — for most organizations the instance is **Worldwide**; sovereign/government clouds (for example US Government GCC High, DoD, and the China cloud) have their own instances with different addresses. You must query the instance that matches your tenant.

> ***Why derive instead of hardcode.*** *Microsoft updates these lists regularly (new ranges, retired ranges, added URLs). Any allow-list you type in by hand is stale the moment an update publishes. The correct operating model is:* ***pull the current set from the web service, transform it into your proxy/firewall's format, and re-pull on a schedule to catch drift.*** *This solution and its scripts are built around that model — they never ship a fixed list of IPs.*

The services that depend on these endpoints

- **Microsoft 365** (Exchange Online, SharePoint Online / OneDrive, Teams, the common/identity area) — the core productivity workloads, served by the endpoint sets above.
- **Microsoft Entra ID** — identity, sign-in, device registration, and Conditional Access. Its destinations sit largely in the **common** service area of the M365 endpoints, plus the Entra / sign-in host names. Nothing device-management-related works if identity is blocked.
- **Microsoft Intune** — device enrollment and management check-in. Intune's required network endpoints (the enrollment and device-management host names and ranges, plus the shared M365 common/identity and content dependencies) are documented by Microsoft as **Intune network endpoints**; treat them as a required set alongside the M365 common area.
- **Windows Autopilot** — provisioning of new devices. Autopilot rides on top of Entra (device registration), Intune (enrollment), and Windows/content endpoints; it has **no separate transport** — if those are allowed, Autopilot works.

• **Microsoft Defender** — endpoint protection cloud services (cloud-delivered protection, sample

submission, and telemetry) have their own documented endpoint set that must be reachable for full protection.

• **Windows Update / content delivery** — the update and content-download endpoints that let managed

devices actually receive updates and app content.

The practical rule: build the allow-list from **(a)** the M365 web service **Optimize + Allow + required** endpoints for your instance, plus **(b)** the documented **Intune**, **Autopilot dependency**, and **Defender** endpoints, and treat that union as the required set for cloud-managed devices behind your filter.

The allow-list lifecycle

An allow-list is not a one-time build; it is a maintained object. This solution runs it as a loop:

1. **Derive** — pull the current required IP/Port/URL set from the Microsoft web service (for your

instance) plus the documented Intune/Autopilot/Defender endpoints. Produce a single normalized **required-set** document.

1. **Build** — turn the required set into concrete **allow-list entries / proxy exceptions** on the

proxy or firewall you administer. Name the objects consistently (this solution specifies the names) so they are auditable and reusable.

1. **Validate** — from a device inside the filtered network, confirm each required destination is

actually reachable on its required port(s). Reachability, not "we added it," is the pass criterion.

1. **Pilot** — apply and validate on a small pilot scope first, and confirm the real workflows

(enrollment, policy sync, update) succeed end to end before touching the fleet.

1. **Expand** — roll the validated allow-list out to the broader environment through change control.

2. **Keep current** — on a schedule, re-pull the web service, compare against the version you built

from, and generate the **delta** (added/removed endpoints) so you update the allow-list *before* drift causes failures. Never let the live list silently diverge from Microsoft's published set.

Deployment model

This solution follows the standard staged model with pass/fail gates at each transition:

Assess → **Configure** → **Pilot** → **Validate** → **Expand.**

• **Assess** — confirm prerequisites (Requirements Guide), inventory the current filter state, and

derive the current required set.

• **Configure** — build the named allow-list entries / proxy exceptions from the required set.

• **Pilot** — apply to a named pilot group only.

• **Validate** — prove reachability and prove the real workflows succeed on the pilot (Testing Guide).

• **Expand** — extend to the fleet, then operate the keep-current loop.

The five PowerShell scripts included map onto this loop: **Discovery** (derive the required set and capture current filter state), **Validation** (per-endpoint reachability), **Report** (allow-list coverage / gap analysis), **Remediation** (generate allow-list updates — preview first), and **Set-ScopeTargeting** (build the pilot target group — preview first).

Requirements Guide

Confirm every requirement below before you build or change an allow-list. These are **in addition to** the category-level prerequisites in `global-requirements-network.md` (Entra tenant, Intune, admin roles, DNS/egress fundamentals, a pilot group, and change control) — this guide covers the prerequisites specific to **egress allow-list / required-endpoint management**.

Each requirement is written as **Description / Why / Verify / Implement if missing.**

R1. A proxy or firewall you administer, with a default-deny egress posture

Description: An outbound proxy, secure web gateway, or firewall through which managed devices reach the internet, where you (the operator) can create and edit allow-list rules / URL categories / address objects. The device enforces default-deny egress (only allowed destinations pass).

Why: The entire solution operates by adding **explicit allow entries** for Microsoft's required endpoints on this device. If you cannot edit its rules, you cannot implement the allow-list. If the device is not actually default-deny, the allow-list is not what is letting traffic through and your validation results will be misleading.

Verify:

• Sign in to the proxy/firewall management console with an account that can create/edit outbound

rules or URL allow categories.

• Confirm the default outbound action for the managed-device network is **deny/block** (not allow).

• Confirm you can create a test allow rule and see it take effect.

Implement if missing:

• Obtain an administrator role on the device from whoever owns it. The **exact menus and object types

differ by vendor — defer those to the vendor's official documentation**; this solution specifies *what* to allow and *how to name it*, not the vendor-specific clicks.

• If egress is not currently default-deny and it is supposed to be, that is a separate hardening

project — but note that this solution is only necessary *because* egress is filtered; on a fully open network no allow-list is required.

R2. The Microsoft required-endpoints source (endpoints web service reachable)

Description: Access to the **Microsoft 365 IP Address and URL web service** so you can pull the current required IP/Port/URL set, plus access to Microsoft's documented **Intune network endpoints**, **Autopilot dependencies**, and **Defender endpoints** references.

Why: The allow-list must be **derived from Microsoft's live published set**, never hardcoded. The web service is the authoritative, versioned, machine-readable source; the documentation pages cover the Intune/Autopilot/Defender endpoints that complete the required set.

Verify:

- From the workstation where you will run the Discovery script, confirm you can reach the Microsoft 365 endpoints web service (the scripts call its `endpoints` and `version` operations for your instance — Worldwide for most tenants).
- Confirm you know your tenant's **instance** (Worldwide, or a sovereign/government instance) — the wrong instance returns the wrong addresses.

Implement if missing:

- Allow the management workstation (or the host that runs the scheduled keep-current job) to reach the Microsoft 365 endpoints web service. This is a read-only HTTPS call and can be permitted from an admin subnet.
- Identify your instance from your tenant type; if you are unsure, default to **Worldwide** and confirm against your tenant.

R3. Change control and a documented rollback

Description: An approved change window and a written rollback for allow-list changes (which entries were added, and how to remove or revert them).

Why: Allow-list changes alter what live devices can and cannot reach. A wrong or overly broad entry can either break connectivity or open unintended egress. Every change must be reversible and recorded.

Verify:

- A change record exists for the planned allow-list build/update, listing the named objects to be created (see the Implementation Guide) and the rollback (delete those named objects / revert to the prior ruleset snapshot).
- The prior ruleset is snapshotted/exported before any change.

Implement if missing:

- Export the current ruleset from the proxy/firewall as a rollback baseline.
- Write the change record using the object names this solution specifies (so add and rollback both reference the same names). Get it approved before the change window.

R4. A pilot scope (small, representative device set)

Description: A small set of devices — behind the same filter — used to validate the allow-list before fleet rollout, plus a named **Entra security group** that targets them (`SG-EGRESS-Pilot`).

Why: Reachability and real-workflow success (enrollment, sync, update) must be proven on a controlled scope before the allow-list affects every device. The pilot is where the pass/fail gate lives.

Verify:

- Entra → **Groups** → confirm `SG-EGRESS-Pilot` exists (or will be created by the `EGRESS-Set-ScopeTargeting.ps1` script) and contains representative pilot devices/users.
- The pilot devices sit behind the same proxy/firewall as production (same egress path).

Implement if missing:

- Create the pilot group with `EGRESS-Set-ScopeTargeting.ps1` (defaults to preview / `-WhatIf`; run with `-Commit` to create), or manually via Entra → **Groups** → **New group** → **Security** named `SG-EGRESS-Pilot`. Add a handful of representative devices that traverse the filtered egress path.

R5. Entra / Intune reachability from the management workstation

Description: The workstation you run the scripts from can reach Microsoft Entra and Intune (the Microsoft Graph endpoints) with an admin account, and has the Microsoft Graph PowerShell modules available for the scripts that read tenant scope.

Why: `EGRESS-Set-ScopeTargeting.ps1` reads/writes the pilot group in Entra, and validation cross-checks against the tenant. Without Graph reachability and rights, the scope/targeting steps cannot run.

Verify:

- `https://entra.microsoft.com` and `https://intune.microsoft.com` load from the workstation with your admin account.
- `Connect-MgGraph` succeeds with the scopes the scripts request (`Group.ReadWrite.All`, `GroupMember.ReadWrite.All`, `Device.Read.All`).

Implement if missing:

- Install the Microsoft Graph PowerShell SDK: `Install-Module Microsoft.Graph -Scope CurrentUser`.
- Ensure the admin account holds a role that can manage groups (for example Groups Administrator or Intune Administrator) and that the management workstation's own egress permits the Graph endpoints (these are themselves part of the required set — see the Implementation Guide).

Implementation Guide

This guide gives the exact, ordered steps to **derive** the required IP/Port/URL set, **build** the allow-list / proxy exceptions, **validate** reachability on a pilot, **expand** to the fleet, and **keep it current** as Microsoft's endpoints change. It follows the staged model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a pass/fail gate at each transition.

Object names used throughout are fixed — **create and reuse them verbatim** so the allow-list, the change records, the reports, and the rollback all refer to the same things:

Object	Name to use
Entra pilot security group	`SG-EGRESS-Pilot`
Derived required-set file	`egress-required-set.json`
Proxy/firewall allow group / policy	`AllowList-M365-Endpoints`
Address/URL object group (IPs + URLs)	`M365-Required-Endpoints`
Discovery output folder	`.\EGRESS-Output\`
Coverage/gap report file	`egress-coverage-report.html`
Proposed allow-list update file	`egress-allowlist-update.txt`
Keep-current baseline (last built version)	`egress-baseline-version.json`

All scripts live in `.\scripts\`. Read-only scripts (Discovery, Validation, Report) make no changes. Change-generating scripts (Remediation, Set-ScopeTargeting) **default to preview /** `-WhatIf` and must be re-run with the commit switch to act.

Phase 0 — Assess (prerequisites and baseline)

Step 0.1 — Confirm prerequisites. Work through `02-requirements.md` (R1–R5) and `global-requirements-network.md`. Do not proceed until each is verified.

Step 0.2 — Snapshot the current filter state (rollback baseline). Export the current outbound ruleset from your proxy/firewall (the vendor's export/backup function — defer the exact steps to the vendor documentation) and store it with your change record. This is your rollback point (R3).

Gate 0 (pass/fail): All prerequisites verified **and** a current-ruleset export saved. Fail → stop and remediate before continuing.

Phase 1 — Derive the required set (Configure, part 1)

Step 1.1 — Run Discovery to pull the current required endpoints. From the management workstation (R2/R5):

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Discovery.ps1 -Instance Worldwide -OutputPath .\
```

This calls the Microsoft 365 IP/URL **web service** (`endpoints` + `version` operations) for your instance, and writes:

- `egress-required-set.json` — the normalized required set (each entry: URLs, IPs, TCP/UDP ports,

serviceArea, category, required flag).

- `egress-baseline-version.json` — the web-service version you built from (used later for drift).
- A current-filter-state summary if you supplied an exported ruleset via `-CurrentRulesetPath`.

Use the instance that matches your tenant (`Worldwide` for most; a sovereign instance otherwise).

Step 1.2 — Add the Intune / Autopilot / Defender endpoints. The web service covers Microsoft 365 (including the common/identity area Entra uses). The Discovery script also emits, in `egress-required-set.json`, a section listing the **documented Intune, Autopilot-dependency, and Defender** endpoint host names/ports that are not fully covered by the web service so they are part of one required-set document. Review that section against Microsoft's current Intune/Defender endpoint documentation and confirm nothing your tenant uses is missing.

Step 1.3 — Scope the required set to what you need. Keep every entry marked **required = true** and every entry in category **Optimize** or **Allow**. Category **Default** entries can be permitted or sent to your general internet path per your policy. Do not narrow below the required + Optimize + Allow set — that is what causes silent failures.

Gate 1 (pass/fail): `egress-required-set.json` exists, contains the M365 required/Optimize/Allow endpoints for the correct instance, and includes the Intune/Autopilot/Defender section. Fail → re-run Discovery / correct the instance.

Phase 2 — Build the allow-list / proxy exceptions (Configure, part 2)

Step 2.1 — Generate the proposed allow-list entries (preview).

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Remediation.ps1 -RequiredSetPath .\EGRESS-Output
```

This runs in **preview / `-WhatIf` by default**: it writes `egress-allowlist-update.txt` — a human-readable list of the URL and IP/CIDR + port entries that should exist in the allow-list — and prints a summary. It changes nothing on the device. Use `-Format` to shape the output for your device (a generic list you paste, a hosts/URL list, or a CIDR list); the exact import into the device is vendor-specific (defer to the vendor docs).

Step 2.2 — Create the named allow-list objects on the proxy/firewall. In the device console, create:

- An address/URL object group named `M365-Required-Endpoints` containing the URLs and IP/CIDR

ranges from `egress-allowlist-update.txt`, with the required TCP/UDP ports (typically 443, and 80 for specific content/CRL/OCSP endpoints as listed).

- An allow rule / policy named `AllowList-M365-Endpoints` that permits the managed-device network

to reach `M365-Required-Endpoints` on those ports, ahead of the default-deny rule.

Match the object names verbatim. The vendor's exact object type and import mechanism are deferred to its documentation; the **content and names** come from this solution.

Step 2.3 — For SSL-inspecting proxies, add decryption bypass for the required set. If the proxy performs SSL/TLS inspection, several Microsoft endpoints (notably certificate-pinned services) must be **bypassed from decryption**, not just allowed. Add the same `M365-Required-Endpoints` set to the proxy's decryption-bypass/exception list. (Which endpoints require bypass is noted in the required-set `notes`; the proxy's bypass mechanism is vendor-specific.)

Gate 2 (pass/fail): `AllowList-M365-Endpoints` and `M365-Required-Endpoints` exist on the device and contain the entries from `egress-allowlist-update.txt`; SSL-bypass added where

required. Fail → correct the objects before piloting.

Phase 3 — Pilot (apply to a limited scope)

Step 3.1 — Build the pilot group (preview, then commit).

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Set-ScopeTargeting.ps1 -PilotGroupName SG-EGRESS
```

Runs in preview / `-WhatIf` by default (shows what it would create/add). Re-run with `-Commit` to create `SG-EGRESS-Pilot` and add the pilot devices.

Step 3.2 — Point the allow-list at the pilot first. Scope `AllowList-M365-Endpoints` to the pilot devices' network/subnet (or apply it to the pilot devices only) so the change is contained. Keep the rest of the fleet on the pre-change state until validation passes.

Gate 3 (pass/fail): `SG-EGRESS-Pilot` populated and the allow-list applied only to the pilot scope. Fail → fix scoping before validating.

Phase 4 — Validate reachability and real workflows

Step 4.1 — Per-endpoint reachability from a pilot device. On a pilot device (behind the filter), run:

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Validation.ps1 -RequiredSetPath .\EGRESS-Output\
```

This tests each required destination on its required port(s) (TCP connect / TLS handshake / HTTP reachability as appropriate) and writes a per-endpoint pass/fail result. **Every required endpoint must be reachable.**

Step 4.2 — Coverage / gap report.

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Report.ps1 -RequiredSetPath .\EGRESS-Output\egre
```

Produces `egress-coverage-report.html`: which required endpoints are covered/reachable and which are gaps. Zero gaps in the required + Optimize + Allow set is the target.

Step 4.3 — Prove the real workflows. On the pilot, confirm end-to-end (see the Testing Guide for the exact gates): a device **enrolls / completes Autopilot**, Intune **policy/compliance syncs**, and an **update / app install succeeds**. Reachability alone is necessary but not sufficient — the workflows are the real proof.

Gate 4 (pass/fail): all required endpoints reachable (Step 4.1), zero required-set gaps in the report (Step 4.2), and enrollment + sync + update succeed on the pilot (Step 4.3). Any fail → use the Troubleshooting Guide to identify the missing endpoint, add it (back to Phase 2), and re-validate.

Phase 5 — Expand to the fleet

Step 5.1 — Broaden the allow-list scope. Through change control (R3), extend `AllowList-M365-Endpoints` from the pilot scope to the full managed-device network. Do not create new entries here — expansion is a scope change, not a content change (the content was validated on the pilot).

Step 5.2 — Spot-validate on the wider fleet. Re-run `EGRESS-Validation.ps1` from one or two production devices in different sites to confirm the same reachability holds fleet-wide.

Gate 5 (pass/fail): allow-list applied fleet-wide and spot-validation passes. Fail → roll back to pilot scope (delete/revert per the change record) and investigate.

Phase 6 — Keep current (operate the loop)

Microsoft changes the published endpoints regularly. Operate a scheduled drift check so you update the allow-list **before** a change breaks connectivity.

Step 6.1 — Schedule a drift check. On a schedule (Microsoft typically publishes changes with lead time; a weekly check is a reasonable default), re-run Discovery and compare to your baseline:

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Discovery.ps1 -Instance Worldwide -OutputPath .\
```

If the web-service **version** is unchanged, there is nothing to do. If it changed, Discovery writes a refreshed `egress-required-set.json` and flags that a delta exists.

Step 6.2 — Generate the delta and update.

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESS-Remediation.ps1 -RequiredSetPath .\EGRESS-Output
```

In preview by default, this reports the **added** and **removed** endpoints since your baseline and writes them to `egress-allowlist-update.txt`. Apply the additions to `M365-Required-Endpoints` (and remove retired entries) through change control, re-validate the changed entries (Phase 4), then update `egress-baseline-version.json` to the new version so the next check compares against it.

Step 6.3 — Record and repeat. Log each drift check and any changes made. A stable required-set with only occasional, expected deltas means the allow-list is healthy.

Gate 6 (ongoing): the live allow-list never silently diverges from the current Microsoft published set; deltas are applied under change control and re-validated. This is the steady state.

Testing Guide

Testing an egress allow-list has two layers: **(1) reachability** — each required destination is allowed through the filter on its required port(s); and **(2) real-workflow success** — the things that were failing (enrollment, sync, update) now work end to end. An allow-list can pass reachability and still miss a workflow if a needed endpoint was mis-scoped, so **both layers must pass** before you expand.

Every gate below is **pass/fail**. Run the reachability tests from a device that sits **behind the same filter** as the managed fleet — testing from an unfiltered admin box proves nothing.

Gate T1 — Required-set derived correctly

Test: Open `egress-required-set.json` (from `EGRESS-Discovery.ps1`). **Pass:** It contains the M365 endpoints for the **correct instance**, all entries marked **required = true** and all **Optimize/Allow** categories are present, and the Intune / Autopilot / Defender section is present. **Fail:** Wrong instance, missing required/Optimize/Allow entries, or missing the Intune/Defender section → re-run Discovery with the correct `-Instance` and re-check.

Gate T2 — Per-endpoint reachability (the core reachability gate)

Test: From a pilot device, run `EGRESS-Validation.ps1` against `egress-required-set.json`. **Pass: Every** required endpoint returns **reachable** on its required port(s) (TCP connect / TLS handshake / HTTP response succeeds). Result file shows zero failures in the required set. **Fail:** Any required endpoint is unreachable → that endpoint is not allowed (or is being decrypted when it must be bypassed). Note the failing URL/IP+port, fix the allow-list (Implementation Phase 2), and re-test.

Gate T3 — Coverage / gap report clean

Test: Run `EGRESS-Report.ps1`; open `egress-coverage-report.html`. **Pass: Zero gaps** in the required + Optimize + Allow set (every required endpoint is both present in the allow-list and reachable). **Fail:** Any required-set gap listed → add the missing entry and re-run T2/T3.

Gate T4 — Enrollment / Autopilot succeeds

Test: Enroll (or run Autopilot provisioning on) a pilot device that reaches the internet **only** through the filtered egress path. **Pass:** The device completes Entra device registration and Intune enrollment (or Autopilot completes to the desktop) with no network-caused stalls or timeouts. **Fail:** Enrollment stalls or errors → use the Troubleshooting Guide to map the stall to the missing endpoint (device-registration / enrollment / identity), add it, and re-test.

Gate T5 — Policy / compliance sync succeeds

Test: On the enrolled pilot device, force an Intune sync (Company Portal / Settings sync, or `Sync` from the Intune admin center) and confirm assigned configuration/compliance/app policies land. **Pass:** The device check-in completes and assigned policies apply; Intune shows the device syncing and reporting status (not stuck "pending"). **Fail:** Sync never completes or policies stay pending → the device-management check-in endpoint is likely blocked; add it and re-test.

Gate T6 — Update / app delivery succeeds

Test: Trigger a Windows Update check and an Intune app install on the pilot device. **Pass:** Update detection/download and the app install both complete (content downloads succeed). **Fail:** Update or content download hangs/fails → a content/update endpoint is blocked; add it and re-test.

Gate T7 — Defender cloud protection reachable (if in scope)

Test: On the pilot device, confirm Defender cloud-delivered protection is active/connected. **Pass:** Defender reports cloud protection connected and reachable. **Fail:** Defender cloud protection cannot reach its service → add the Defender endpoints and re-test.

Gate T8 — Drift check operates

Test: Run the keep-current Discovery with `-BaselineVersionPath` (Implementation Phase 6). **Pass:** The version comparison runs, correctly reports "no change" or produces a delta, and the delta (if any) can be previewed via `EGRESS-Remediation.ps1`. **Fail:** The drift check errors or does not detect a known change → fix the scheduled job / baseline path before relying on it.

Overall exit criteria (before Expand)

All of the following on the pilot, behind the real filter:

- **T1–T3** pass: required set correct, every required endpoint reachable, zero coverage gaps.
- **T4–T6** pass: enrollment/Autopilot, policy sync, and update/app delivery all succeed.
- **T7** passes if Defender is in scope.
- **T8** passes: the drift check is working.

Only when all applicable gates pass do you proceed to fleet **Expand** (Implementation Phase 5).

Troubleshooting Guide

Egress-filtering failures share a signature: the device behaves as if a Microsoft service is "missing," with no firewall error shown on the device. The diagnostic method is always the same — **map the symptom to the Microsoft service it needs, then map that service to the required endpoint, then prove reachability of that endpoint** with `EGRESS-Validation.ps1`. This guide gives that map.

Diagnostic method (use for every symptom)

1. **Reproduce behind the filter.** Confirm the affected device reaches the internet only through the proxy/firewall under management (not a secondary path). A failure that disappears off the filter confirms an allow-list gap.
1. **Run reachability.** From the affected device run `EGRESS-Validation.ps1` against `egress-required-set.json`. The FAIL rows name the exact host+port that is blocked.
1. **Confirm coverage.** Run `EGRESS-Report.ps1`; the GAP rows are the required endpoints to add.
2. **Add and re-validate.** Add the missing entry to `M365-Required-Endpoints` (regenerate the proposal with `EGRESS-Remediation.ps1` if helpful), then re-run validation until zero gaps.
1. **Re-test the workflow** (Testing Guide gates T4–T7).

If reachability passes for everything but a workflow still fails, suspect **SSL inspection** (a certificate-pinned endpoint being decrypted instead of bypassed) or the **wrong instance** in the required set.

Symptom: Enrollment or Autopilot stalls / errors

What it looks like: A new device sits at a provisioning/enrollment screen, times out, or errors during Autopilot; the device may register in Entra but never reach Intune, or never register at all.

Which service/endpoint is likely missing:

- **Entra device registration / identity** — the common/identity service area from the M365 web service and the device-registration host (`enterpriseregistration.windows.net`). If identity is blocked, nothing downstream works.
- **Intune enrollment / management** — the Intune management host (`*.manage.microsoft.com`).
- **Autopilot deployment service** — the Autopilot dependency hosts, plus (because Autopilot rides on them) Entra + Intune + Windows content endpoints.

Fix: Validate those hosts on 443; add any FAIL to the allow-list; if the proxy inspects SSL, add the identity/enrollment hosts to the decryption-bypass list. Re-run enrollment (Gate T4).

Symptom: Device shows enrolled but policies never sync / stay "pending"

What it looks like: Intune lists the device as enrolled, but configuration/compliance/app policies never apply; sync appears to run but nothing lands, or check-in status is stuck pending.

Which service/endpoint is likely missing:

- **Intune device-management check-in** (`*.manage.microsoft.com`) on 443 — this is the sync channel.
- **The M365 common/identity area** — tokens for the sync are obtained via identity; if identity is intermittently blocked, sync silently fails.

Fix: Validate `*.manage.microsoft.com` and the common/identity endpoints; add any gap; re-run a forced sync (Gate T5). If sync only fails from one site, the gap is site-specific — validate from a device at that site.

Symptom: Windows Update or app install fails / hangs

What it looks like: Update detection or download stalls, apps fail to install with content-download errors, or delivery never completes — despite enrollment and sync working.

Which service/endpoint is likely missing:

- **Windows Update / content-delivery endpoints** (`*.update.microsoft.com`, `*.delivery.mp.microsoft.com`, `*.windowsupdate.com`) on 443 and 80. Content endpoints commonly need **port 80** in addition to 443 — a 443-only allow-list is a frequent cause of this exact symptom.

Fix: Validate the update/content hosts on **both** 443 and 80; add the missing port entries (`M365-Required-Endpoints` must include 80 for these); re-run an update check and app install (Gate T6).

Symptom: Defender cloud protection disconnected

What it looks like: Defender reports cloud-delivered protection off/unreachable, or sample submission fails, while other services work.

Which service/endpoint is likely missing:

- **Defender cloud endpoints** (`*.wd.microsoft.com`, `*.wdcp.microsoft.com`,

`*.endpoint.security.microsoft.com`, and related) on 443.

Fix: Validate the Defender hosts; add gaps; confirm decryption bypass for any pinned Defender endpoint; re-check cloud protection (Gate T7).

Symptom: Everything intermittently breaks after "no change on our side"

What it looks like: Services that worked start failing across the board without a local change.

Likely cause: Microsoft changed the published endpoints and the allow-list drifted out of date, or the required set was built from the **wrong instance**.

Fix:

- Run the drift check: `EGRESS-Discovery.ps1 -BaselineVersionPath ...`. If it reports **changed**,

generate the delta with `EGRESS-Remediation.ps1 -BaselineRequiredSetPath ...`, apply the additions, and re-validate.

- Confirm `-Instance` matches your tenant (Worldwide vs a sovereign cloud). A wrong instance yields

addresses your tenant never uses and omits the ones it does.

Symptom: Validation passes but a workflow still fails

Likely cause: SSL/TLS inspection decrypting a certificate-pinned endpoint (the TCP connect succeeds, so validation passes, but the pinned handshake fails inside the app), or an endpoint allowed by IP but requested by a URL the proxy categorizes elsewhere.

Fix: Add the affected endpoints to the proxy's **decryption-bypass** list (not just the allow-list); ensure URL-based allow entries exist for URL-filtering proxies, not only IP entries. Re-run the workflow gate.

Script-level issues

Problem	Cause	Fix
`EGRESS-Discovery.ps1` cannot reach the web service	The management host's own egress blocks the endpoints web service	Allow the web-service host from the admin subnet (R2); the management host is itself part of the required set.
Wrong addresses in the required set	Wrong `-Instance`	Re-run with the instance matching your tenant.

Problem	Cause	Fix
`EGRESS-Validation.ps1` all-FAIL from an admin box	Tested off the filtered path	Run it **on a device behind the same proxy/firewall** as the fleet.
`EGRESS-Set-ScopeTargeting.ps1` cannot create the group	Not connected to Graph, or missing rights	Install Microsoft.Graph, `Connect-MgGraph` with the listed scopes, ensure a group-management admin role (R5).
Remediation "changed nothing on the device"	By design	The script only writes a proposal file; import it into the proxy/firewall per the vendor's documentation.

Best Practices

Operating an egress allow-list for Microsoft's cloud-managed endpoints well comes down to two disciplines: **rigorous change control** and **routine drift review**. The list is a living object; it succeeds or fails on how it is maintained, not on how it was first built.

Change control

• **Never edit the live allow-list ad hoc.** Every add/remove goes through the change record that names the objects (`AllowList-M365-Endpoints`, `M365-Required-Endpoints`) and the rollback. Ad-hoc edits are how the list silently diverges from Microsoft's published set.

• **Snapshot before you change.** Export the current ruleset before any edit so rollback is a restore, not a reconstruction from memory.

• **Preview before you commit.** Always generate the proposed change (`EGRESS-Remediation.ps1` runs in preview / `-WhatIf` by default) and review `egress-allowlist-update.txt` before touching the device. Same for scope changes (`EGRESS-Set-ScopeTargeting.ps1` previews by default).

• **Pilot every content change.** New or changed endpoints get validated on `SG-EGRESS-Pilot` first (reachability **and** the enrollment/sync/update workflows) before fleet expansion. Scope expansion is a separate, later change from content change.

• **Use the fixed object names verbatim.** Consistent naming makes the allow-list, reports, change records, and rollback all refer to the same objects — essential for audit and handover.

Derive, never hardcode

• **Always build from the Microsoft web service for your instance**, not from a static list you typed or copied. Confirm the instance (Worldwide vs a sovereign cloud) matches your tenant every time.

• **Keep the required + Optimize + Allow set intact.** Do not trim below it to "tighten" — that is the

direct cause of silent enrollment/sync/update failures. Category **Default** endpoints are where you have discretion.

- **Record the baseline version** (`egress-baseline-version.json`) you built from, so every future change is a reviewable delta against a known point, not a fresh guess.

Drift review

- **Schedule the drift check** (Implementation Phase 6). A weekly Discovery-with-baseline run is a reasonable default; Microsoft usually publishes changes with lead time, so a regular cadence catches them before they bite.
- **Act on the version, not the calendar.** If the web-service version is unchanged, do nothing. When it changes, generate the delta, apply additions and remove retired entries under change control, re-validate the changed entries, then advance the baseline.
- **Trend the deltas.** A healthy environment shows small, occasional, expected deltas. A surprise — large delta, or reachability failing without a published change — points at a local proxy/firewall change and should be investigated as drift on *your* side.

SSL inspection and least privilege

- **Bypass decryption for endpoints that require it.** Certificate-pinned Microsoft services fail if inspected; keep them on the decryption-bypass list, not just the allow-list.
- **Scope the allow-list to the managed-device network**, not "any source." The Microsoft endpoints should be reachable by the devices that need them, on the required ports only (typically 443, with 80 only for the specific content/CRL/OCSP endpoints listed).
- **Keep the management workstation's own egress in the required set.** The host that runs Discovery and the drift job needs the web-service and Graph endpoints allowed — treat it like a managed device.

Documentation and handover

- **Keep the required-set file, reports, and change records together.** `egress-required-set.json`, `egress-coverage-report.html`, and the change history are the audit trail; store them with the rollback baseline.
- **Document the instance, the schedule, and the object names** so the next operator can run the loop without reverse-engineering it.

Deliverables Summary

This solution is a complete, repeatable operating package for administering the IP/Port/URL allow-lists and proxy/firewall exceptions that let Microsoft 365, Intune, Autopilot, Defender, and Entra required endpoints work through tight egress filtering — derived from Microsoft's live published set, validated, and kept current.

Documents

Document	What it gives you
`01-solution-overview.md`	Why cloud-managed endpoints need egress access; the Microsoft required-endpoints model (M365 IP/URL web service, Intune, Autopilot, Defender, Entra); the allow-list lifecycle and staged deployment model.
`02-requirements.md`	Every prerequisite (proxy/firewall you administer, the required-endpoints source, change control, a pilot scope, Entra/Intune reachability) with Description / Why / Verify / Implement-if-missing.
`03-implementation.md`	The ordered build: derive the required set, build the named allow-list / proxy exceptions, pilot, validate, expand, and operate the keep-current drift loop — with a pass/fail gate at each phase.
`05-testing.md`	Pass/fail gates: required-set correctness, per-endpoint reachability, coverage/gap clean, and real-workflow success (enrollment/Autopilot, policy sync, update/app delivery, Defender).
`06-troubleshooting.md`	Symptom-to-endpoint mapping — enrollment stalls, sync fails, update fails — tracing each failure to the missing endpoint and the fix.
`07-best-practices.md`	Change control, derive-never-hardcode, and drift review disciplines that keep the list healthy over time.
`global-requirements-network.md`	Category-level prerequisites shared by all Network solutions.
`executive-summary.md`	One-page summary for decision-makers.

PowerShell scripts (`.\scripts\`)

Script	Role	Safety
`EGRESS-Discovery.ps1`	Pulls the current required IP/Port/URL set from the Microsoft 365 web service (for your instance) plus Intune/Autopilot/Defender endpoints; captures current filter state; supports baseline-version drift detection.	Read-only

Script	Role	Safety
`EGRESS-Validation.ps1`	Tests per-endpoint reachability of each required IP/Port/URL from a device behind the filter.	Read-only
`EGRESS-Report.ps1`	Produces an allow-list coverage / gap report from the required set and validation results.	Read-only
`EGRESS-Remediation.ps1`	Generates the allow-list update entries (and drift deltas) for import into the proxy/firewall.	Preview / `-WhatIf` by default
`EGRESS-Set-ScopeTargeting.ps1`	Builds/populates the `SG-EGRESS-Pilot` Entra group for staged rollout.	Preview / `-WhatIf` by default

Named objects you will create (verbatim)

- Entra pilot group: `SG-EGRESS-Pilot`
- Proxy/firewall allow rule: `AllowList-M365-Endpoints`
- Address/URL object group: `M365-Required-Endpoints`
- Derived required set: `egress-required-set.json`
- Baseline version: `egress-baseline-version.json`
- Coverage report: `egress-coverage-report.html`
- Proposed update: `egress-allowlist-update.txt`

What "done" looks like

Behind your real filter, on the pilot: the required set is derived for the correct instance, every required endpoint is reachable, the coverage report shows zero gaps, and enrollment/Autopilot, policy sync, and update/app delivery all succeed — after which you expand to the fleet and run the scheduled drift check so the live allow-list never silently diverges from Microsoft's published set.

Important notes

- **Vendor-neutral on the proxy/firewall.** This solution specifies *what* to allow and *how to name it*; the exact console steps for your specific proxy/firewall are deferred to that vendor's documentation.
- **No hardcoded IPs.** The required set is always derived live from Microsoft's web service — the scripts never ship a fixed list of addresses, because Microsoft's endpoints change.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EGRESS-Discovery.ps1

Purpose: Pulls the current required IP/Port/URL set from the Microsoft 365 IP Address and URL web service for your tenant instance, adds the documented Intune, Autopilot, and Defender endpoints, and writes one normalized required-set file. Optionally compares the web-service version against a saved baseline to detect drift. Read-only: it changes nothing on any device. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESS-Discovery.ps1 -Instance Worldwide -OutputPath .\EGRESS-Output Needs outbound HTTPS to the Microsoft 365 endpoints web service. No admin rights required. Add -BaselineVersionPath to detect drift; add -CurrentRulesetPath to summarize an exported ruleset. EXPECTED RESULTS Writes egress-required-set.json and egress-baseline-version.json to the output folder and prints a summary (endpoint-set count, required/Optimize/Allow counts, drift status). PASS = required-set file written for the correct instance; FAIL = web service unreachable or wrong instance. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESS-Discovery.ps1 -Instance Worldwide -OutputPath .\EGRESS-Output Needs outbound HTTPS to the Microsoft 365 endpoints web service. No admin rights required. Add -BaselineVersionPath to detect drift; add -CurrentRulesetPath to summarize an exported ruleset. EXPECTED RESULTS Writes egress-required-set.json and egress-baseline-version.json to the output folder and prints a summary (endpoint-set count, required/Optimize/Allow counts, drift status). PASS = required-set file written for the correct instance; FAIL = web service unreachable or wrong instance. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

EGRESS-Remediation.ps1

Purpose: Turns the required-set file (EGRESS-Discovery.ps1) into a human-readable allow-list update that you import into your proxy or firewall. If a baseline version is supplied it reports the added/removed endpoints (drift delta) instead of the full list. Runs in preview by default and writes a proposal file only; it never contacts or changes any proxy/firewall device. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESS-Remediation.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -Format generic Preview is the default. Add -Commit only to acknowledge the proposal is final; even with -Commit this script only writes the proposal file (the actual device import is vendor-specific). EXPECTED RESULTS Writes egress-allowlist-update.txt (URLs + IP/CIDR + ports to allow) and prints a summary. PASS = proposal file written for review; you then import it into the device per its documentation. .NOTES White-label — the repeatable way. Preview/-WhatIf by default for any change-generating action. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESS-Remediation.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -Format generic Preview is the default. Add -Commit only to acknowledge the proposal is final; even with -Commit this script only writes the proposal file (the actual device import is vendor-specific). EXPECTED RESULTS Writes egress-allowlist-update.txt (URLs + IP/CIDR + ports to allow) and prints a summary. PASS = proposal file written for review; you then import it into the device per its documentation. .NOTES White-label — the repeatable way. Preview/-WhatIf by default for any change-generating action. #>

EGRESS-Report.ps1

Purpose: Combines the required-set file (EGRESS-Discovery.ps1) with the validation results (EGRESS-Validation.ps1) into a single coverage/gap report showing which required endpoints are reachable and which are gaps. Read-only: it reads the two input files and writes a report; it changes nothing on any device or allow-list. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESS-Report.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -ValidationResultsPath .\EGRESS-Output -OutputPath .\EGRESS-Output No internet access or admin rights required; operates on local input files. EXPECTED RESULTS Writes egress-coverage-report.html (and a JSON summary) and prints a coverage summary. PASS = zero gaps in the required + Optimize + Allow set; FAIL = one or more required endpoints are gaps. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESS-Report.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -ValidationResultsPath .\EGRESS-Output -OutputPath .\EGRESS-Output No internet access or admin rights required; operates on local input files. EXPECTED RESULTS Writes egress-coverage-report.html (and a JSON summary) and prints a coverage summary. PASS = zero gaps in the required + Optimize + Allow set; FAIL = one or more required endpoints are gaps. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

EGRESS-Set-ScopeTargeting.ps1

Purpose: Creates the Entra security group SG-EGRESS-Pilot (if missing) and adds the pilot device or user object IDs you supply, so the allow-list can be validated on a limited scope before fleet rollout. Runs in preview by default and shows exactly what it would create/add; it makes no changes to Entra until you re-run with -Commit. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESS-Set-ScopeTargeting.ps1 -PilotGroupName SG-EGRESS-Pilot -MemberObjectId id-one,id-two Needs Microsoft Graph PowerShell and admin rights (scopes Group.ReadWrite.All, GroupMember.ReadWrite.All, Device.Read.All). Preview by default; add -Commit to apply. EXPECTED RESULTS Preview prints the group and members it would create/add. With -Commit it creates/populates the group and prints the group id. PASS = SG-EGRESS-Pilot exists and contains the pilot members. .NOTES White-label — the repeatable way. Preview/-WhatIf by default for any change-generating action. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESS-Set-ScopeTargeting.ps1 -PilotGroupName SG-EGRESS-Pilot -MemberObjectId id-one,id-two Needs Microsoft Graph PowerShell and admin rights (scopes Group.ReadWrite.All, GroupMember.ReadWrite.All, Device.Read.All). Preview by default; add -Commit to apply. EXPECTED RESULTS Preview prints the group and members it would create/add. With -Commit it creates/populates the group and prints the group id. PASS = SG-EGRESS-Pilot exists and contains the pilot members. .NOTES White-label — the repeatable way. Preview/-WhatIf by default for any change-generating action. #>

EGRESS-Validation.ps1

Purpose: Reads the required-set file produced by EGRESS-Discovery.ps1 and tests each required endpoint for reachability on its required port(s) using DNS resolution and TCP/TLS connect tests. Run it from a device that sits behind the same proxy/firewall as the managed fleet. Read-only: it makes no changes to any device or allow-list. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESS-Validation.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -OutputPath .\EGRESS-Output Run on a pilot device behind the filter. Needs the egress path under test; no admin rights required. EXPECTED RESULTS Writes egress-validation-results.json and prints a per-endpoint

PASS/FAIL table. PASS = every required endpoint reachable on its required port(s); FAIL = one or more required endpoints unreachable. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESS-Validation.ps1 -RequiredSetPath .\EGRESS-Output\egress-required-set.json -OutputPath .\EGRESS-Output Run on a pilot device behind the filter. Needs the egress path under test; no admin rights required. EXPECTED RESULTS Writes egress-validation-results.json and prints a per-endpoint PASS/FAIL table. PASS = every required endpoint reachable on its required port(s); FAIL = one or more required endpoints unreachable. .NOTES White-label — the repeatable way. Read-only for discovery/validation. #>